Pertemuan 2

Moda Self-Study: Berbagai Permasalahan dalam Graf

Disusun Untuk Memenuhi Tugas Mata Kuliah Praktikum Terapan Teori Graf

Dosen Pengampu:

TIM DOSEN

DISUSUN OLEH:

MUHAMMAD RAFLY ROMEO NASUTION

10123875

3KA25

FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI

SISTEM INFORMASI

UNIVERSITAS GUNADARMA

ATA 2025/2026

Soal:

Pada praktikum minggu depan, Anda akan mulai mencoba berbagai algoritma sebagai salah satu solusi pemecahan masalah dengan menggunakan Teori Graf. Silahkan Anda membuat sebuah Case Study permasalahan pada domain Anda yang dapat dipecahkan menggunakan Teori Graf. Contoh:

Case Study: Bagaimana menemukan jalur terpendek untuk menuju suatu tempat?

Context Question:

- Directed/Undirected? Directed.
- Weighted/Unweighted? Weighted.
- Sparse/Dense? Kemungkinan Dense.

Karakteristik masalah sesuai dengan tipe? Shortest-Path-Problem.

Jawaban:

Case Study:

Pemerintah kota ingin mengoptimalkan jaringan transportasi di wilayahnya dengan tujuan agar kendaraan dapat mencapai lokasi tujuan dengan lebih efisien. Pemerintah memiliki data mengenai berbagai jalan yang menghubungkan beberapa lokasi di kota tersebut, termasuk jarak antar jalan dan kondisi lalu lintas.

Permasalahan yang dihadapi adalah bagaimana menentukan jalur perjalanan yang paling efisien sehingga waktu tempuh dan biaya transportasi dapat diminimalkan. Masalah ini dapat dimodelkan menggunakan **Teori Graf**, di mana:

- **Simpul (vertex)** merepresentasikan lokasi atau persimpangan jalan.
- **Sisi (edge)** merepresentasikan jalan yang menghubungkan dua lokasi.
- **Bobot (weight)** merepresentasikan jarak atau waktu tempuh antar lokasi.

Dengan model graf tersebut, kita dapat menggunakan **algoritma Dijkstra** untuk mencari jalur dengan jarak atau waktu tempuh paling pendek dari satu titik ke titik tujuan.

Context Question:

- **Directed / Undirected?**

  Directed, karena dalam beberapa kondisi jalan dapat memiliki arah tertentu atau aturan lalu lintas yang berbeda.

- **Weighted / Unweighted?**

  Weighted, karena setiap jalan memiliki jarak atau waktu tempuh yang berbeda.

- **Sparse / Dense?**

  Kemungkinan Dense, karena dalam jaringan transportasi kota banyak lokasi yang saling terhubung.

- **Karakteristik masalah sesuai dengan tipe?**

  Shortest Path Problem.

Contoh Implementasi dengan Python:

```
graph = {
    'A': {'B': 5, 'C': 3},
    'B': {'D': 4},
    'C': {'B': 2, 'D': 1},
    'D': {'A': 1, 'G': 2},
    'E': {'D': 1, 'G': 3},
    'F': {'E': 2, 'H': 1},
    'G': {'H': 2},
    'H': {}
}
```

```
def dijkstra(graph, start, end):
    distances = {node: float('inf') for node in
graph}
    distances[start] = 0
    unvisited_nodes = graph.copy()
    visited_nodes = {}

    while unvisited_nodes:
        current_node = min(unvisited_nodes,
key=lambda node: distances[node])

        for neighbor, distance in
graph[current_node].items():
            if neighbor not in unvisited_nodes:
                continue
            new_distance =
distances[current_node] + distance
            if new_distance <
distances[neighbor]:
                distances[neighbor] =
new_distance

        visited_nodes[current_node] =
distances[current_node]
        del unvisited_nodes[current_node]

        if current_node == end:
            break

    return visited_nodes
```

```
start = 'A'
end = 'G'
print(dijkstra(graph, start, end))
```

Output:

Jika program dijalankan dengan titik awal **A** dan tujuan **G**, maka hasilnya:

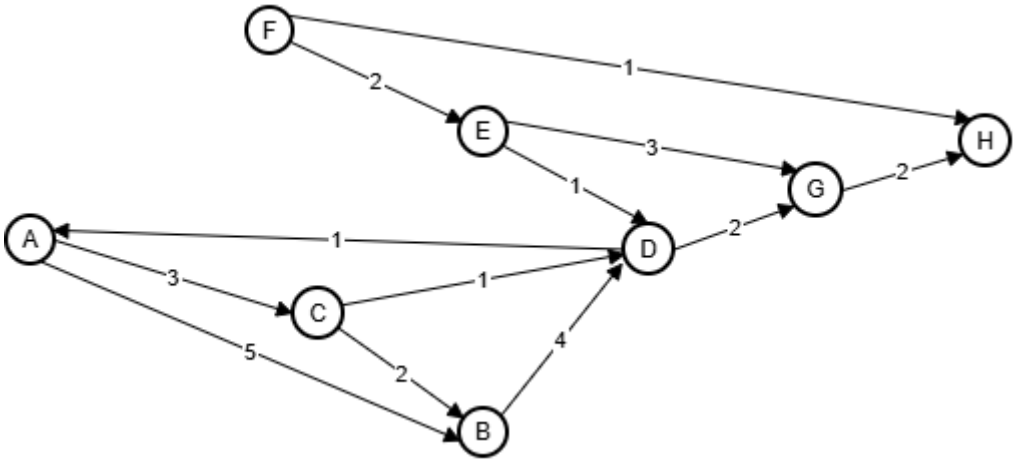
```
PS C:\Users\Acer\Documents\TUGAS KULIAH - Universitas Gunadarma\SEMESTER 5\Praktikum LAB SI\Pengantar Sai
iversitas Gunadarma\SEMESTER 5\Praktikum LAB SI\Pengantar Sains Data - Python\M2\dijkstra_shortest_path.p
{'A': 0, 'C': 3, 'D': 4, 'B': 5, 'G': 6}
```

Output tersebut menunjukkan jarak terpendek dari node awal **A** menuju node lainnya dalam graf, termasuk jarak terpendek menuju node tujuan **G**.

Visualisasi Graf:

Untuk mempermudah memahami struktur graf yang digunakan dalam implementasi algoritma Dijkstra, graf dapat divisualisasikan dalam bentuk diagram. Visualisasi ini membantu menggambarkan hubungan antar simpul serta bobot pada setiap sisi graf.

Berikut adalah visualisasi graf berdasarkan struktur data yang terdapat pada kode Python:



Penjelasan Visualisasi Graf

Pada diagram graf di atas:

- **Node (A, B, C, D, E, F, G, H)** merepresentasikan simpul atau lokasi dalam graf.
- **Panah (→)** menunjukkan arah hubungan antar simpul sehingga graf bersifat **Directed Graph**.
- **Angka pada sisi graf** menunjukkan **bobot (weight)** yang dapat merepresentasikan jarak, waktu tempuh, atau biaya antar lokasi.

Beberapa contoh hubungan pada graf tersebut antara lain:

- Node **A** terhubung ke **B** dengan bobot **5** dan ke **C** dengan bobot **3**.
- Node **C** memiliki jalur menuju **B** dengan bobot **2** dan menuju **D** dengan bobot **1**.
- Node **D** memiliki jalur menuju **G** dengan bobot **2**.
- Node **G** memiliki jalur menuju **H** dengan bobot **2**.

Graf ini digunakan sebagai representasi jaringan yang akan diproses oleh **algoritma Dijkstra** untuk menentukan jalur terpendek dari node awal menuju node tujuan.

Analisis Jalur Terpendek:

Berdasarkan hasil eksekusi program menggunakan algoritma Dijkstra dengan **node awal A** dan **node tujuan G**, diperoleh output sebagai berikut:

```
{'A': 0, 'C': 3, 'B': 5, 'D': 4, 'G': 6}
```

Hasil tersebut menunjukkan jarak terpendek dari node awal **A** menuju beberapa node lainnya. Nilai **0 pada node A** menunjukkan bahwa jarak dari titik awal ke dirinya sendiri adalah nol. Kemudian algoritma menemukan jalur menuju node lain dengan bobot minimum.

Jika dianalisis dari graf yang telah divisualisasikan, jalur terpendek dari **A menuju G** dapat melalui beberapa node perantara. Salah satu jalur yang ditemukan oleh algoritma adalah:

A → C → D → G

Perhitungan bobot jalur tersebut adalah sebagai berikut:

- A → C = 3
- C → D = 1
- D → G = 2

Total bobot jalur:

3 + 1 + 2 = 6

Sehingga dapat disimpulkan bahwa **jarak terpendek dari node A menuju node G adalah 6**. Jalur ini dipilih karena memiliki total bobot yang paling kecil dibandingkan kemungkinan jalur lainnya dalam graf.

Dengan menggunakan algoritma **Dijkstra**, proses pencarian jalur terpendek dapat dilakukan secara sistematis sehingga membantu menemukan rute paling efisien dalam suatu jaringan.